

CAP 4. CAMADA DE APLICAÇÃO

AULA 19: PROGRAMAÇÃO VIA API SOCKET

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Programação via Sockets

Meta da aula

- Ensinar como construir aplicações cliente/servidor que se comunicam usando sockets

Socket API

- Introduzido no BSD4.1 UNIX, 1981
- Seguem o paradigma cliente/servidor
- Dois tipos de serviço de transporte via socket API:
 - Datagrama não confiável
 - Confiável e orientado a fluxo de bytes

socket

Uma interface controlada pelo S.O., local ao host, que é criada e controlada pela aplicação

Uma porta na qual o processo de aplicação pode enviar e receber dados de outros processo de aplicação

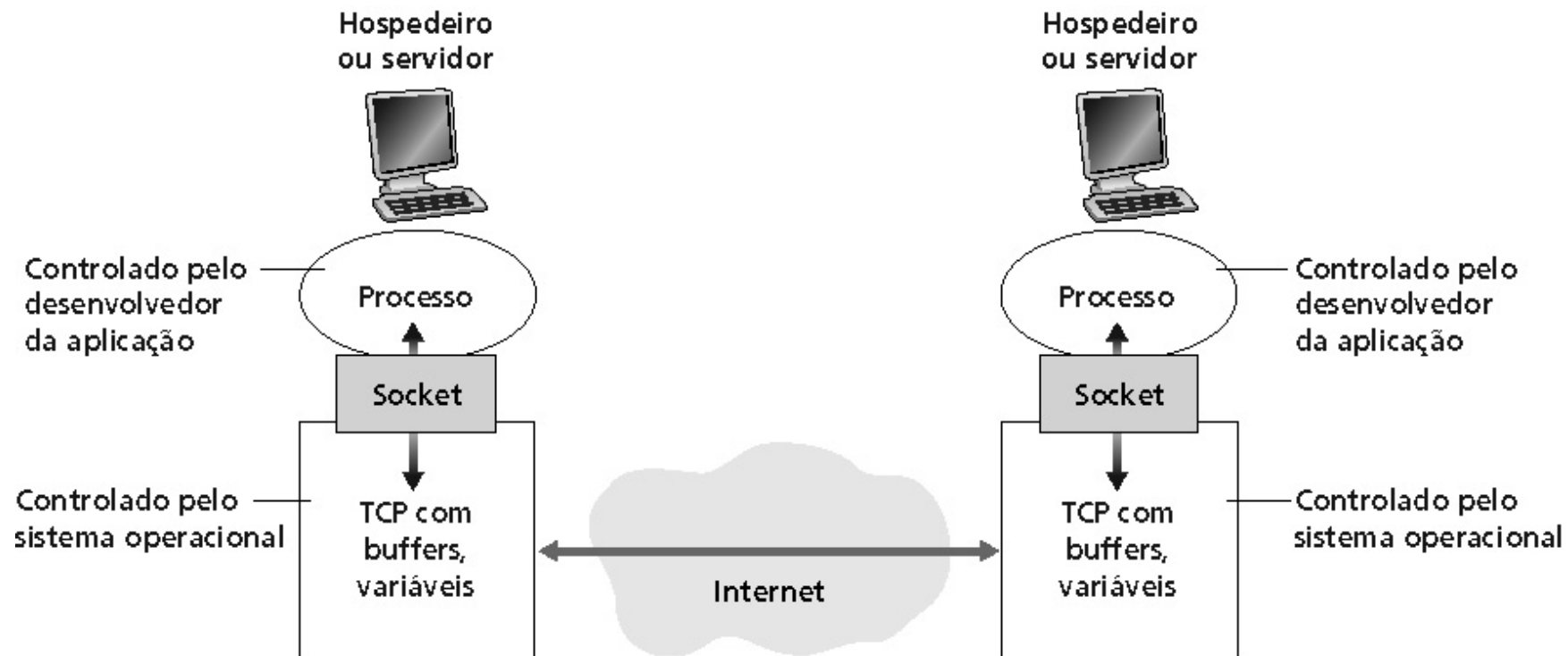
Programação de Socket usando TCP

Socket

- uma porta entre o processo de aplicação e o protocolo de transporte fim-a-fim (TCP ou UDP)

Serviço TCP

- transferência confiável de bytes de um processo para outro



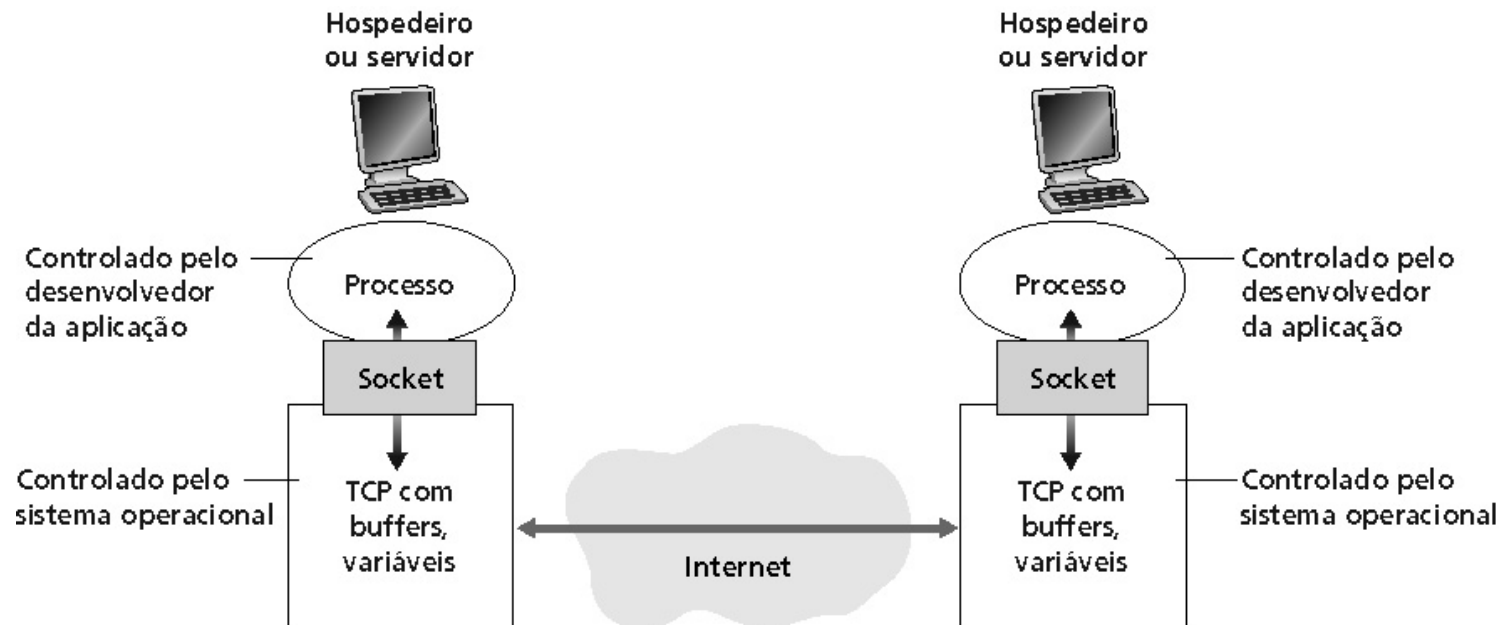
Programação de Socket usando TCP

Cliente deve contactar servidor

- processo servidor deve antes estar em execução
- servidor deve antes ter criado socket (porta) que aguarda contato do cliente

Cliente cria socket TCP:

- socket TCP local ao cliente
- especificar endereço IP, número de porta do processo servidor



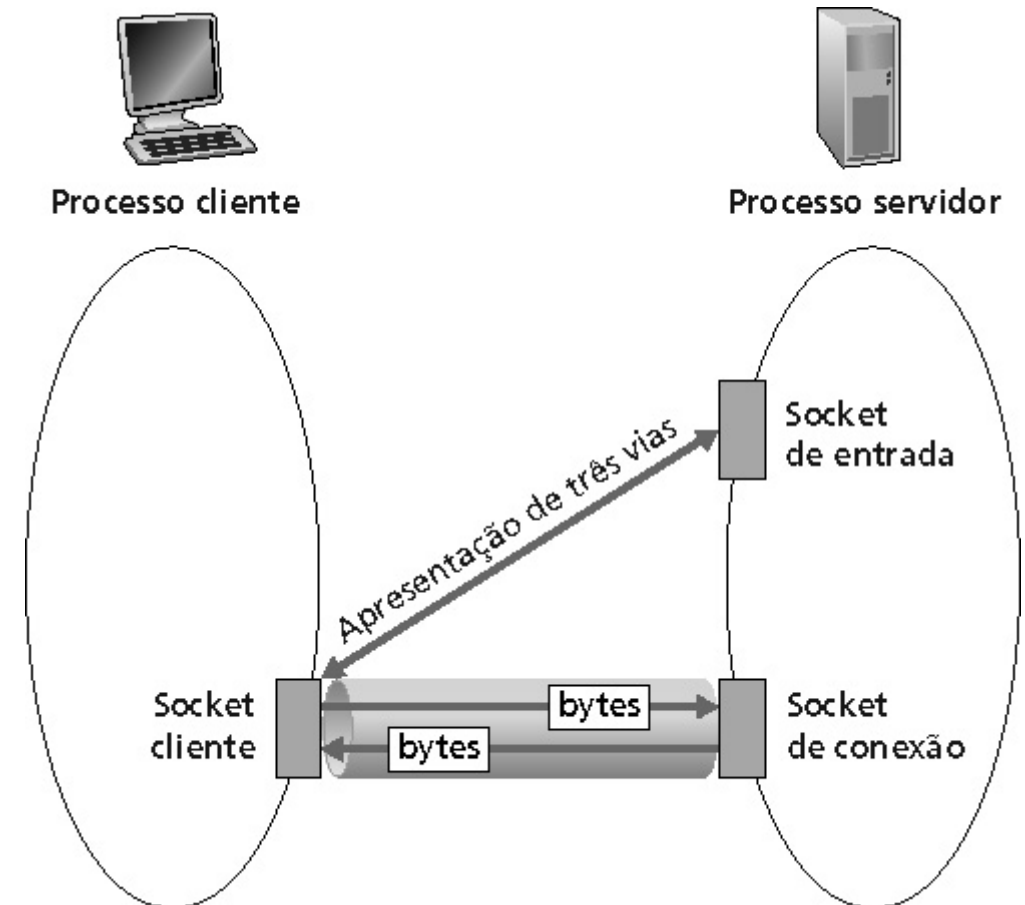
Programação de Socket usando TCP

Quando cliente cria socket

- TCP do cliente estabelece conexão com TCP do servidor

Quando contatado pelo cliente, o TCP do servidor

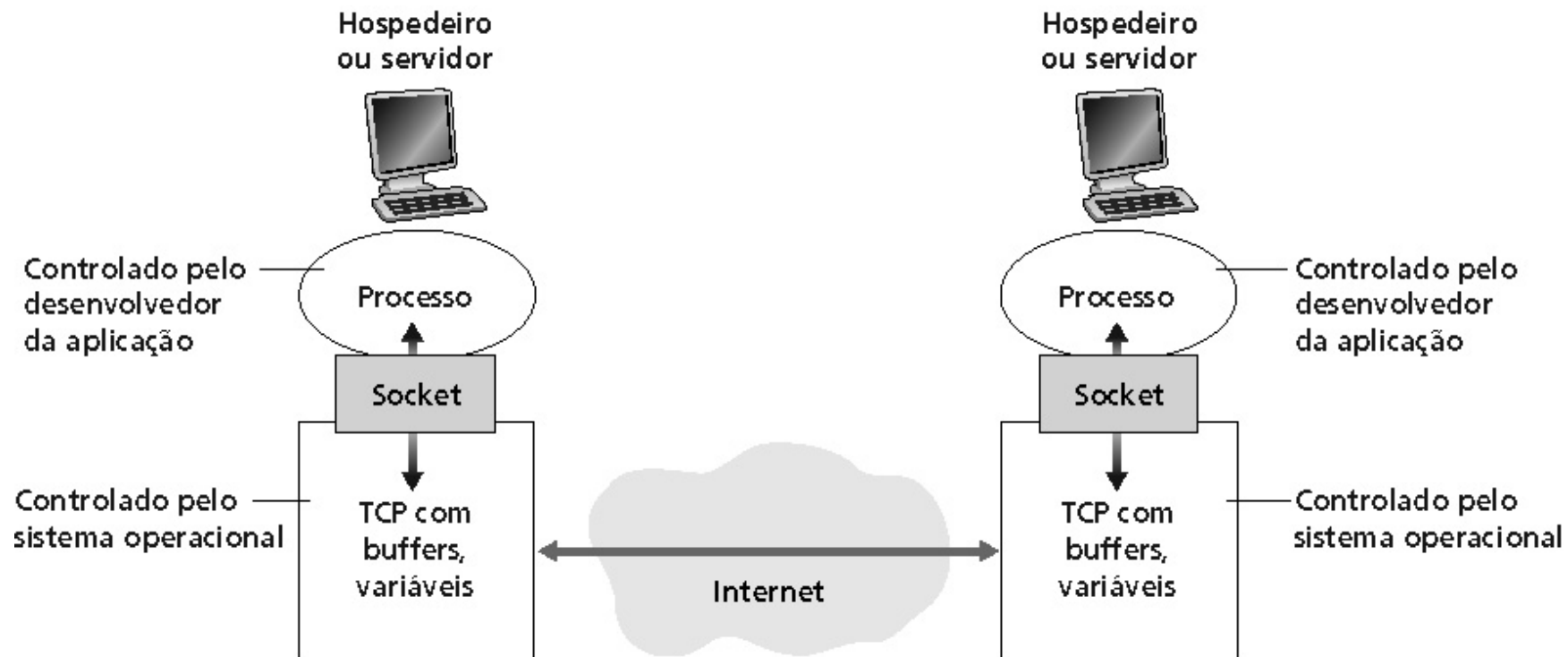
- cria socket novo para que o processo servidor possa se comunicar com o cliente
- permite que o servidor converse com múltiplos clientes



Programação de Socket usando TCP

Do ponto de vista da aplicação

- TCP provê transferência confiável, ordenada de bytes (“tubo”) entre cliente e servidor



Jargão stream

Stream

- é uma sequência de caracteres que fluem para dentro ou para fora de um processo

Stream de entrada

- é agregado a alguma fonte de entrada para o processo, ex.: teclado ou socket

Stream de saída

- é agregado a uma fonte de saída, ex.: monitor ou socket

Exemplo: Cliente e Servidor UDP

Cliente TCP

```
import socket

HOST = "127.0.0.1" # Nome ou end. IP do servidor
PORT = 65432      # Porta usada pelo servidor

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    # socket.AF_INET indica que é um socket IPv4
    # socket.SOCK_STREAM indica que o socket é TCP

    # estabelece a conexão
    s.connect((HOST, PORT))

    message = input("Digite uma string: ").encode('utf-8')
    # envia string codificada
    s.sendall(message)

    #recebe os dados da conexão, até 1024B e decodifica
    data = s.recv(1024).decode('utf-8')

print(data)
```

Servidor TCP

```
import socket

HOST = "127.0.0.1" # localhost
PORT = 65432      # porta alocada

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    # socket.AF_INET indica que é um socket IPv4
    # socket.SOCK_STREAM indica que o socket é TCP

    s.bind((HOST, PORT)) # Liga o socket ao endereço/porta
    s.listen()           # bloqueia aguardando conexão
    conn, addr = s.accept() # Aceita a conexão

    with conn:
        print(f"Connected by {addr}")
        while True:
            data = conn.recv(1024) # leitura de até 1024B
            if not data:
                break
            conn.sendall(data.decode('utf-8').upper().encode('utf-8'))
```

Exemplo: Cliente e Servidor TCP

Cliente UDP

```
import socket

HOST = "127.0.0.1" # Nome ou end. IP do servidor
PORT = 5000        # Porta usada pelo servidor
dest = (HOST, PORT) # Destino da mensagem

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as udp:
    # socket.AF_INET indica que é um socket IPv4
    # socket.SOCK_DGRAM indica que o socket é UDP

    message = input("Digite uma string: ").encode('utf-8')
    udp.sendto(message, dest)
    msg, server = udp.recvfrom(1024)
    print(msg.decode('utf-8'))
    udp.close()
```

Servidor UDP

```
import socket

HOST = "127.0.0.1" # localhost
PORT = 5000        # porta alocada

with socket.socket(socket.AF_INET, SOCK_DGRAM) as udp:
    # socket.AF_INET indica que é um socket IPv4
    # socket.SOCK_DGRAM indica que o socket é UDP

    udp.bind((HOST, PORT)) # Liga o socket ao endereço/porta
    msg, cliente = udp.recvfrom(1024)
    print(f"Received from {cliente}")
    response = msg.decode('utf-8').upper().encode('utf-8')
    udp.sendto(response, cliente)
```

Exemplo: Web Server Python

```
import socket
# Define socket host e porta
SERVER_HOST = '0.0.0.0'
SERVER_PORT = 8000

# Create socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((SERVER_HOST, SERVER_PORT))
server_socket.listen(1)
print('Listening on port %s ...' % SERVER_PORT)

while True:
    # Espera por conexões
    client_connection, client_address = server_socket.accept()

    # obté o pedido do cliente
    request = client_connection.recv(1024).decode()
    # Parse HTTP headers
    headers = request.split('\n')
    command = headers[0].split()
    method = command[0]
    filename = command[1]

    ...
```

```
#Analisa pedido
if method != 'GET':
    response = 'HTTP/1.0 501 NOT IMPLEMENTED\n\nNot
                Implemented Method'
else:
    # Trata no me arquivo
    if filename == '/':
        filename = 'index.html'
    else:
        filename = filename[1:] # Remove caractere "/"
    # Obtém recurso
    try:
        fin = open(filename)
        content = fin.read()
        fin.close()
        # Monta resposta
        response = 'HTTP/1.0 200 OK\n\n' + content
    except FileNotFoundError:
        response = 'HTTP/1.0 404 NOT FOUND\n\nFile Not
                    Found'
    client_connection.sendall(response.encode())
    client_connection.close()

# Fecha socket
server_socket.close()
```

Pontos Importantes

Programação com API Socket

- Conhecimentos gerais para entendimento de como é selecionado e usado o protocolo da camada de transporte